Тема 5

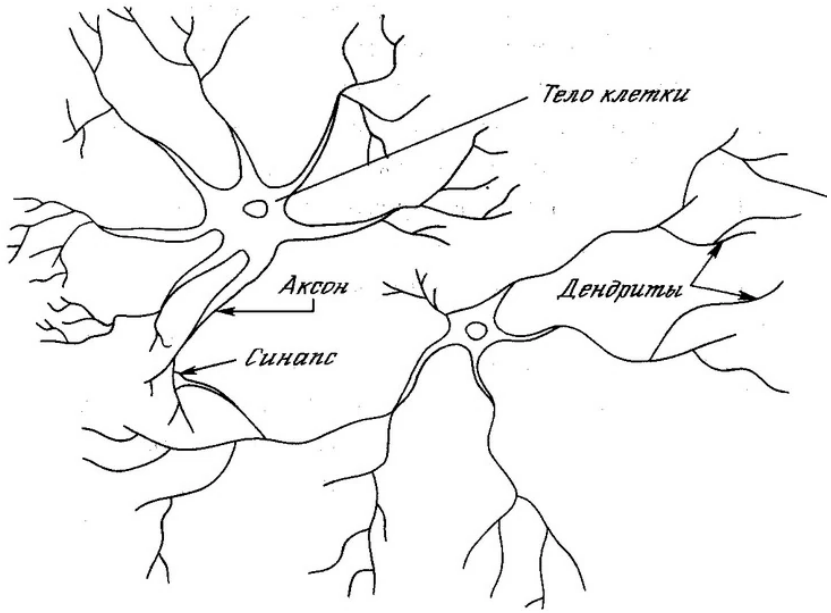
Простейшие нейронные сети

Простейшие нейронные сети

Биологическая подоснова
Формальный нейрон
Многослойный персептрон
Обучение многослойного персептрона
Сеть Хопфилда

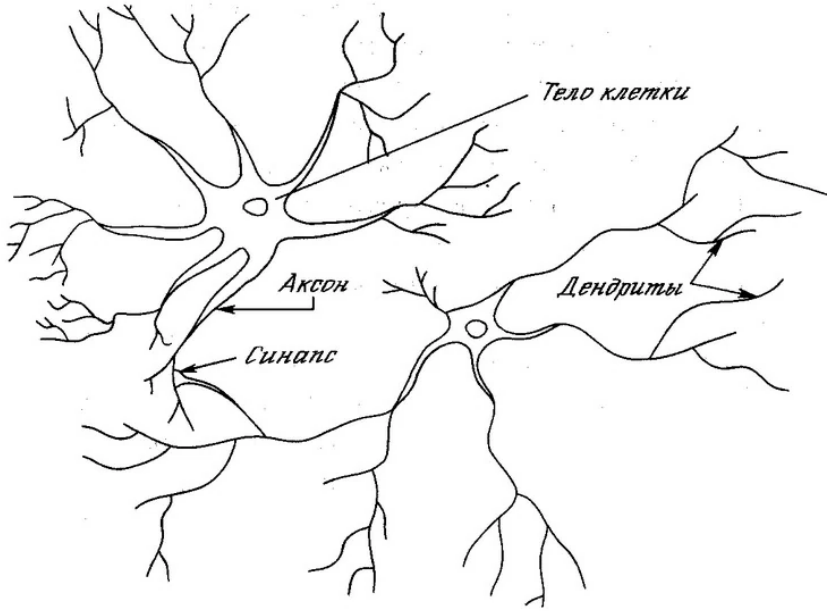
Биологический нейрон

Известно, что нервная система высших животных способна обучаться. Идея: воспроизвести техническими средствами похожий процесс для адаптации алгоритма к имеющимся данным, т.е. для машинного обучения.



Нервные клетки (нейроны) имеют множество выростов, образующих сложную трёхмерную структуру. Нейроны способны передавать друг другу электрические сигналы в форме коротких импульсов — спайков.

Функционирование биологического нейрона



1. нервные клетки касаются друг друга в особых точках — **синапсах**, через которые электрические импульсы одних клеток воздействуют на другие клетки;
2. если синапсы, расположенные на специальных коротких выростах (дендритах), возбуждаются выше некоторого порога, нейрон сам порождает импульс;
3. он проходит по длинному отростку (аксону) и передаётся расположенным на нём синапсам, возбуждая другие нейроны.

Обучение биологического нейрона

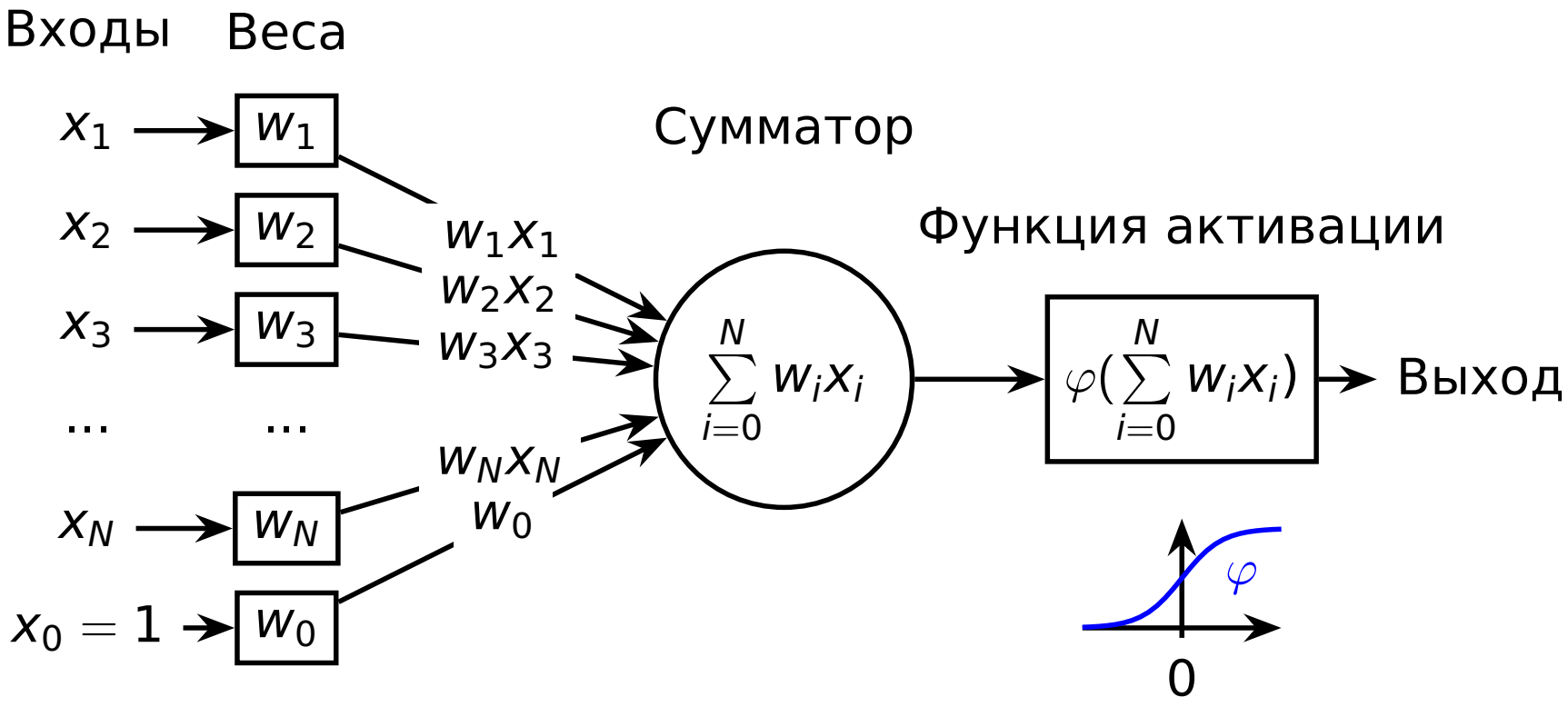
Такая структура способна обучаться путём настройки силы влияния одних клеток на другие. Сигналы через синапс могут передаваться как электрически, так и химически, поэтому делается это модификацией синапсов — изменением химического состава, влияющим на выделение промежуточных соединений, передающих возбуждение.

Правило Хебба: если нейрон A часто активизируется прямо перед нейроном B и между ними существует связь через синапс, эта связь усиливается.

Это означает проявление **ассоциации** нейронов — возбуждения одного под влиянием другого.

Формальный нейрон

Технически возможно реализовать спайковые нейронные сети (такие, в которых возбуждение выражается числом порождаемых спайков), но пока чаще используются сильно упрощённые модели: возбуждение выражается значением сигнала на выходе, которое однозначно определяется текущими входами.



Формальный нейрон (2)

Значение на выходе — линейная комбинация значений на входе, пропущенная через некоторую нелинейную функцию — **функцию активации**. Коэффициенты этой линейной комбинации w_i — **синаптические коэффициенты** или веса.

$$y = \varphi(\sum_{i=1}^{n} w_i x_i)$$

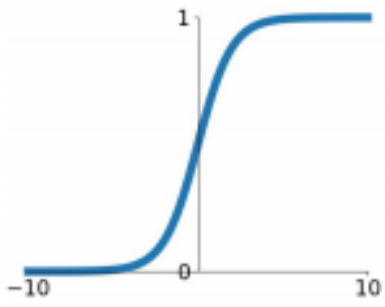
Очень часто специально вводят один фиктивный входной сигнал $x_0 = 1$, чтобы ввести в линейную комбинацию постоянное смещение.

Функция активации

Непрерывная функция одного действительного аргумента.
Типичные варианты:

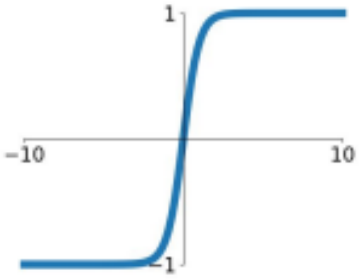
Логистическая функция

$$y = \frac{1}{1 + e^{-x}}$$



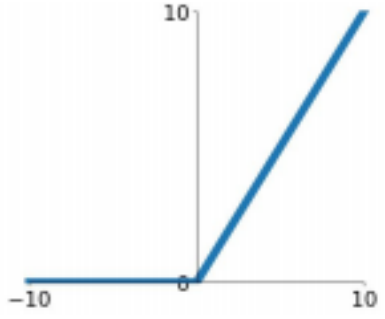
Гиперболический тангенс

$$tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$



ReLU (rectified linear unit)

$$y = max(x, 0)$$



Свойства формального нейрона

Пусть у формального нейрона один вход x. **Вопрос:** как выглядит его уравнение?

Свойства формального нейрона

Пусть у формального нейрона один вход x. **Вопрос:** как выглядит его уравнение?
Ответ: $y = \varphi(w_0 + w_1 x)$

Вопрос: как выглядит его график по сравнению с графиком $\varphi(x)$?

Свойства формального нейрона

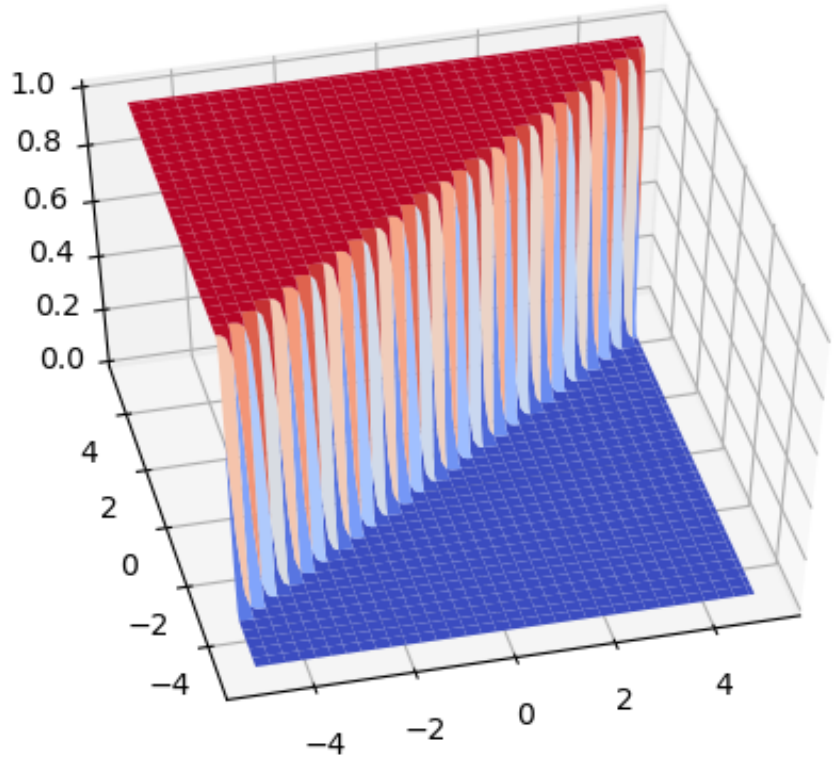
Пусть у формального нейрона один вход x. **Вопрос:** как выглядит его уравнение?
Ответ: $y = \varphi(w_0 + w_1 x)$

Вопрос: как выглядит его график по сравнению с графиком $\varphi(x)$?
Ответ: он сохраняет свой вид, но сдвинут и/или растянут вдоль оси $0X$.

Свойства формального нейрона (2)

Если входов несколько, график также сохранит свой вид, но будет строиться в многомерном пространстве.

Пример графика формального нейрона с двумя входами:



Видно, что такой нейрон работает как линейный классификатор.

Свойства формального нейрона (3)

Вывод: один формальный нейрон с произвольным числом входов n способен линейно разделить пространство $\mathbb{R}^n$ на две части.

Вопрос: не встречались ли нам ранее подобные модели?

Свойства формального нейрона (3)

Вывод: один формальный нейрон с произвольным числом входов n способен линейно разделить пространство $\mathbb{R}^n$ на две части.

Вопрос: не встречались ли нам ранее подобные модели?
Ответ:

- линейный пороговый классификатор; его функция активации $\varphi(x) = sign(x)$;
- логистическая регрессия; её функция активации $\varphi(x) = \frac{1}{1+e^{-x}}$;
- радиальные базисные функции.

Это значит, что один формальный нейрон с логистической функцией активации способен максимально правдоподобно решить задачу линейной классификации.

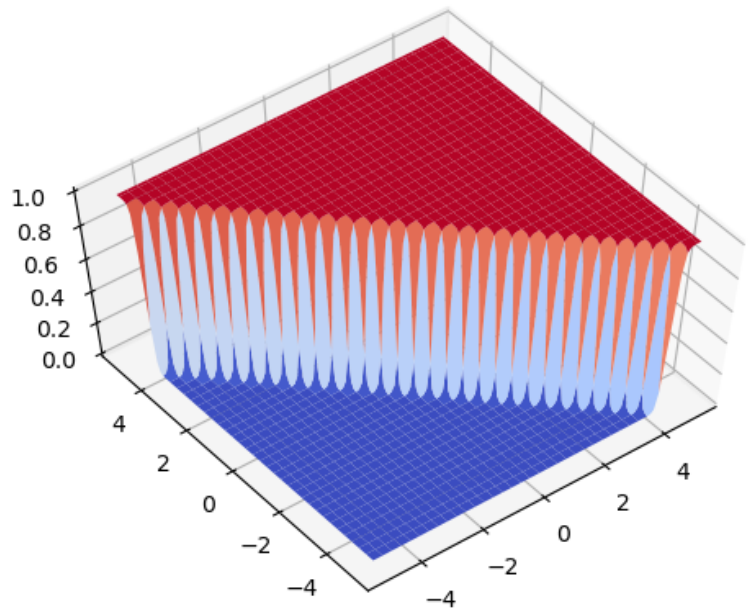
Можно ли скомбинировать несколько нейронов?

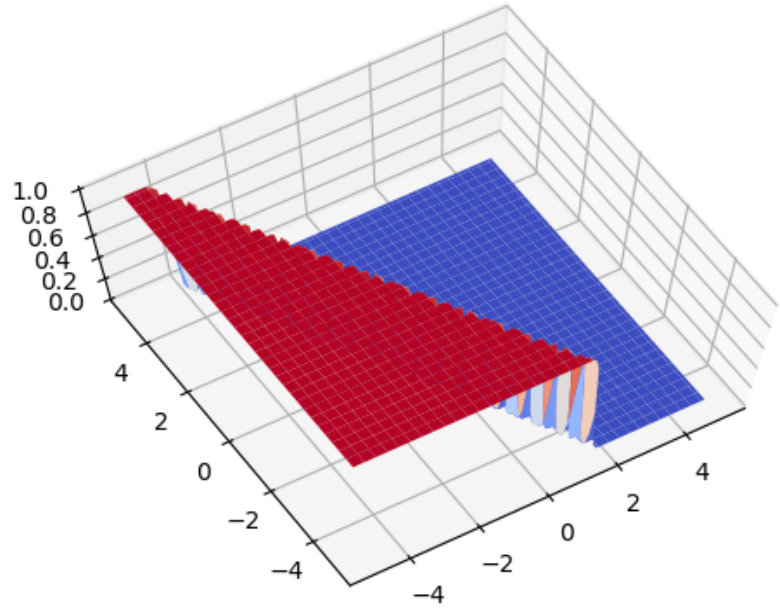
Вопрос: получится ли решать более сложные задачи классификации, если как-то собрать вместе несколько отдельных формальных нейронов?

Можно ли скомбинировать несколько нейронов?

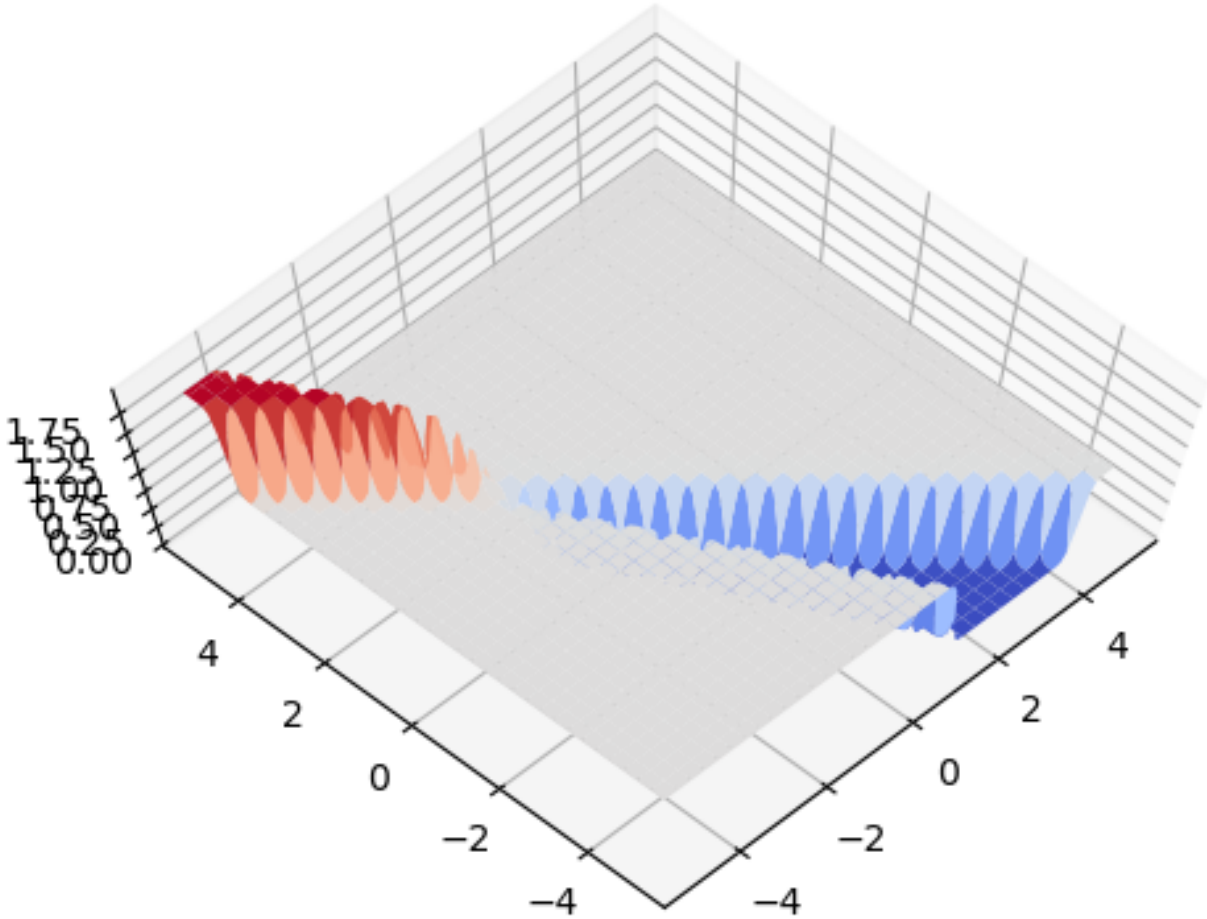
Вопрос: получится ли решать более сложные задачи классификации, если как-то собрать вместе несколько отдельных формальных нейронов?

Ответ: да. Можно сначала обработать исходные данные несколькими нейронами, а потом линейно скомбинировать их выходы.





Результат комбинирования



Теперь можно в зависимости от коэффициентов линейной комбинации использовать либо нижнюю (синюю) часть, либо верхнюю (красную), либо любую из них в комбинации с серой.

Результат комбинирования

Иначе говоря: при решении задач классификации отдельные формальные нейроны способны линейно отделить части выборки друг от друга (каждый по-своему). Они образуют **первый слой**.

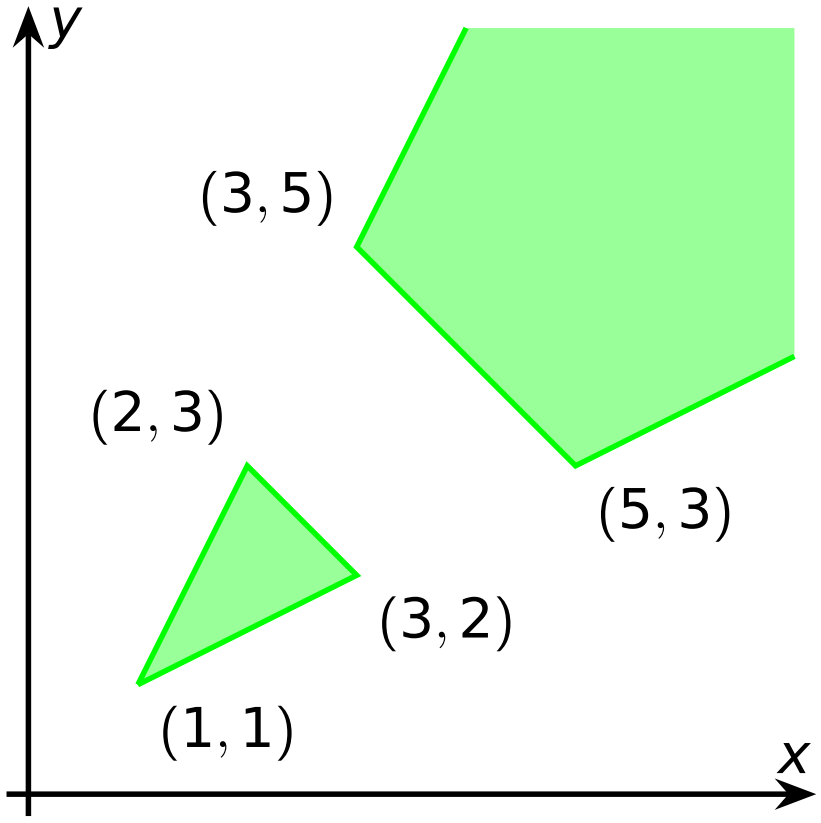
Если подать выходы первого слоя на другие формальные нейроны, они способны собрать из них сплошные области с кусочно-линейными границами. Эти формальные нейроны образуют **второй слой**.

Третий слой способен получить объединения таких областей (т.е. подмножества, по сути, произвольной формы — лишь бы хватило нейронов).

Реально можно использовать и больше слоёв; иногда это позволяет снизить число нейронов в первых слоях.

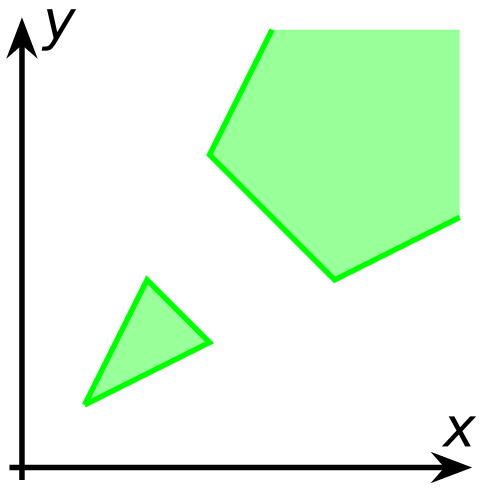
Пример создания сети

Пусть нужно построить нейросеть, различающую двумерные точки, лежащие внутри закрашенной области, и точки вне неё:



Вопрос: сколько входов и выходов будет у этой сети?

Пример: входы и выходы



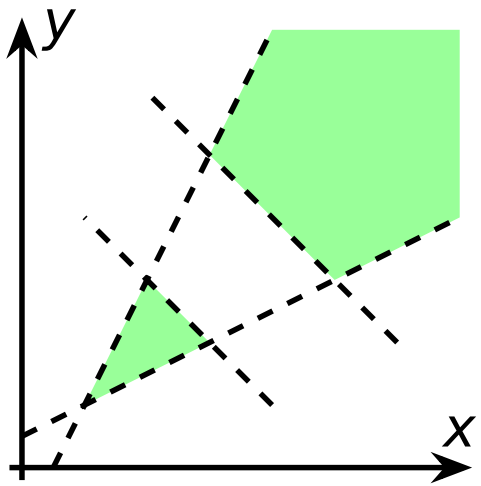
Вопрос: сколько входов и выходов будет у этой сети?
Ответ: два входа — координаты x и y;
Выходов тоже два: один отвечает за распознавание точек внутри области, другой — точек вне области. В идеале точки внутри области должны вызывать реакцию сети «10», точки вне области — «01»; реально это не всегда так: приходится выбирать класс с большим значением выхода.

Для простоты используем нейроны со ступенчатой функцией активации: $\mathbb{S}(x) = 1$, если x>0, иначе 0.

Пример: первый слой

На нейроны первого слоя подаются координаты x и y; это значит, что эти нейроны линейно разделяют области плоскости.

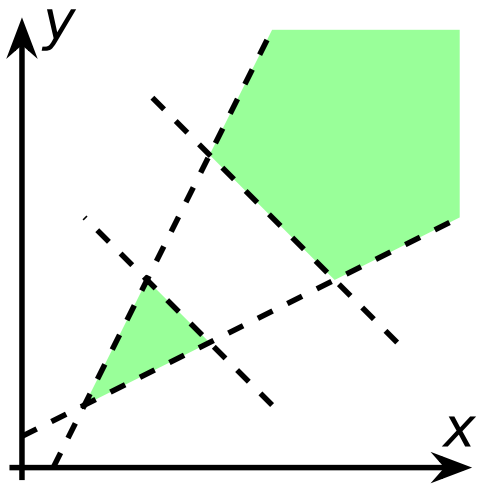
Вопрос: сколько линейных разделений плоскости нужно, чтобы выделить фигуру?



Пример: первый слой

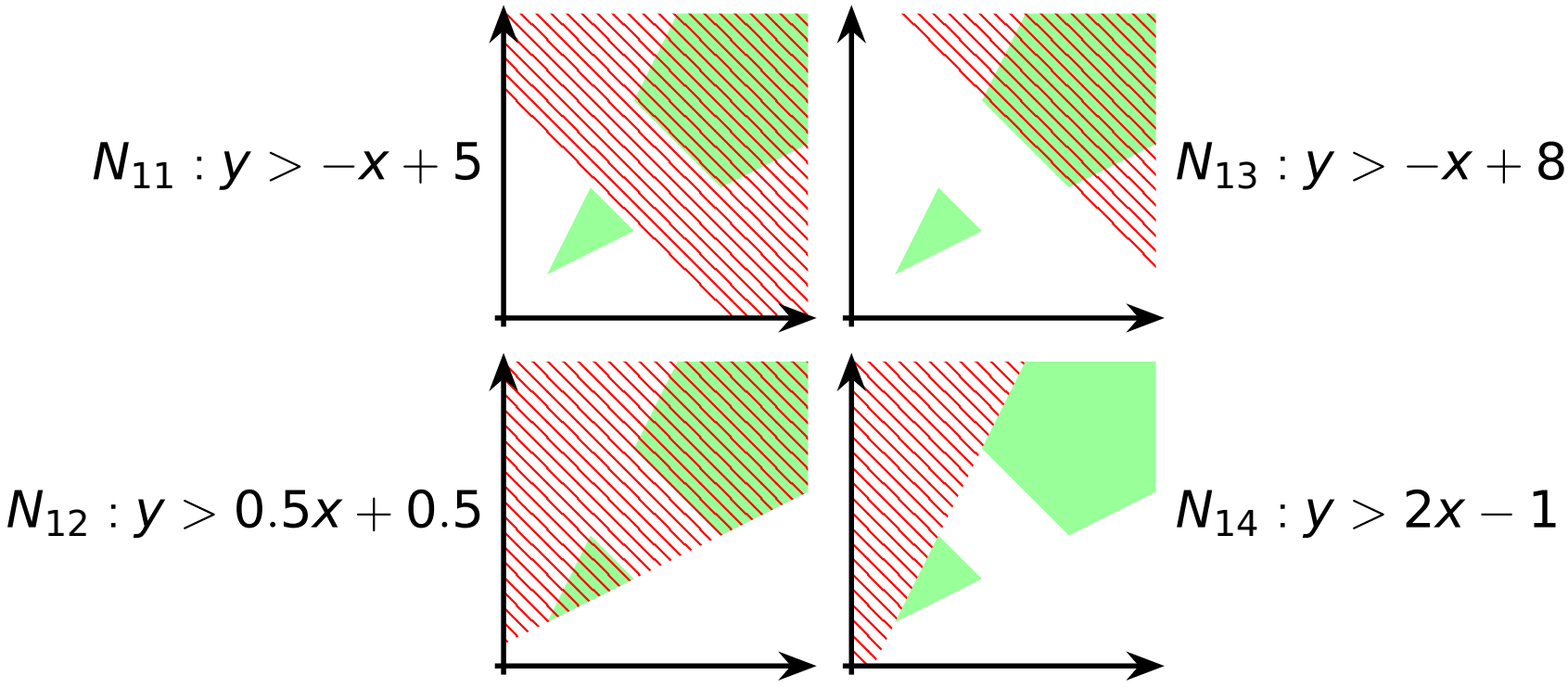
На нейроны первого слоя подаются координаты x и y; это значит, что эти нейроны линейно разделяют области плоскости.

Вопрос: сколько линейных разделений плоскости нужно, чтобы выделить фигуру?



Ответ: для каждой прямой, содержащей сторону фигуры, нужно одно линейное разделение, т.е. один нейрон. Всего их нужно 4.

Пример: первый слой (2)

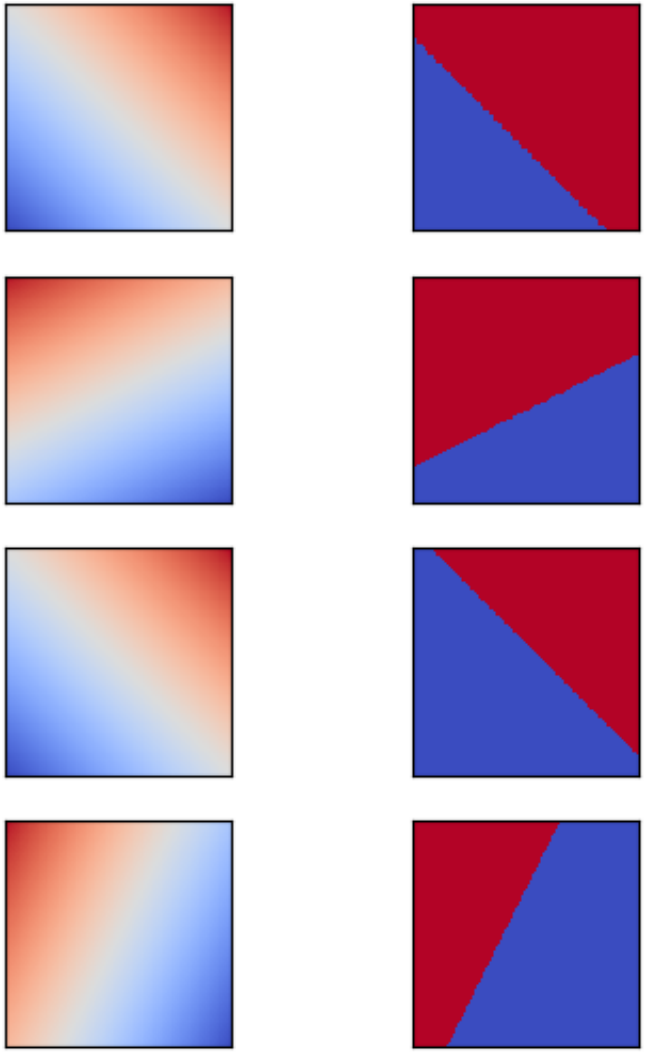


Уравнения нейронов:

$N_{11} = \mathbb{S}(x + y - 5)$, $N_{13} = \mathbb{S}(x + y - 8)$,

$N_{12} = \mathbb{S}(-0.5x + y - 0.5)$, $N_{14} = \mathbb{S}(-2x + y + 1)$

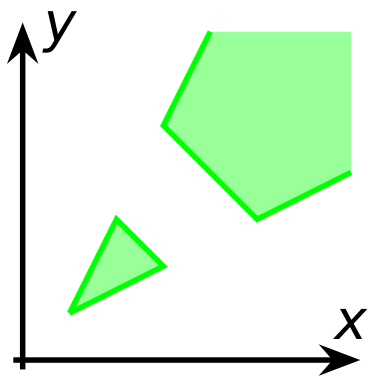
Выходы нейронов первого слоя



Графики построены для диапазонов x и y $[-3, 10]$.
Слева показаны взвешенные суммы каждого нейрона первого слоя до нелинейного преобразования ($\sum w_i x_i$), справа — нелинейные преобразования от них, т.е. выходы нейронов $\mathbb{S}(\sum w_i x_i)$.

Пример: второй слой

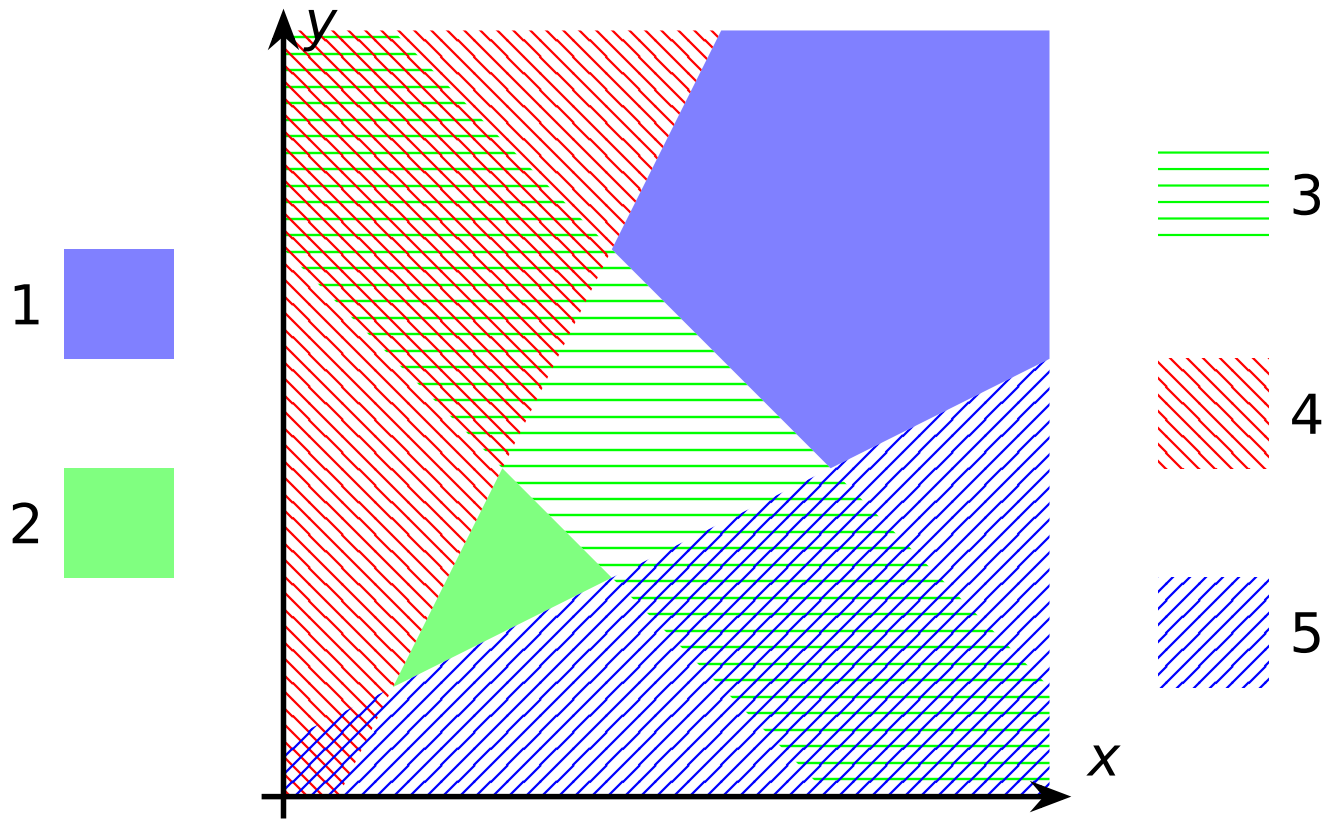
Вопрос: сколько нужно нейронов второго слоя и почему?



Подсказка: каждый нейрон второго слоя отвечает за одну выпуклую область.

Пример: второй слой (2)

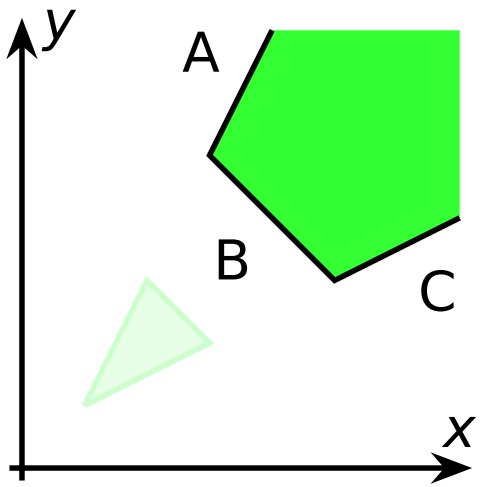
Для распознавания области интереса нужно выделить две выпуклых области, для распознавания других частей — три.



Как должен выглядеть нейрон второго слоя, отвечающий, например, за область 1?

Пример: второй слой (3)

Видно, что граница этой области образована тремя отрезками прямых. Обозначим их A, B, C:



К этим линиям относятся следующие нейроны первого слоя:

- A: N_{14}, внутри области он неактивен, т.е. $N_{14} = 0$, т.е. $1 - N_{14} = 1$;
- B: N_{13}, внутри области он активен, т.е. $N_{13} = 1$;
- C: N_{12}, внутри области он активен, т.е. $N_{12} = 1$.

Итоговые уравнения второго слоя

Если в области

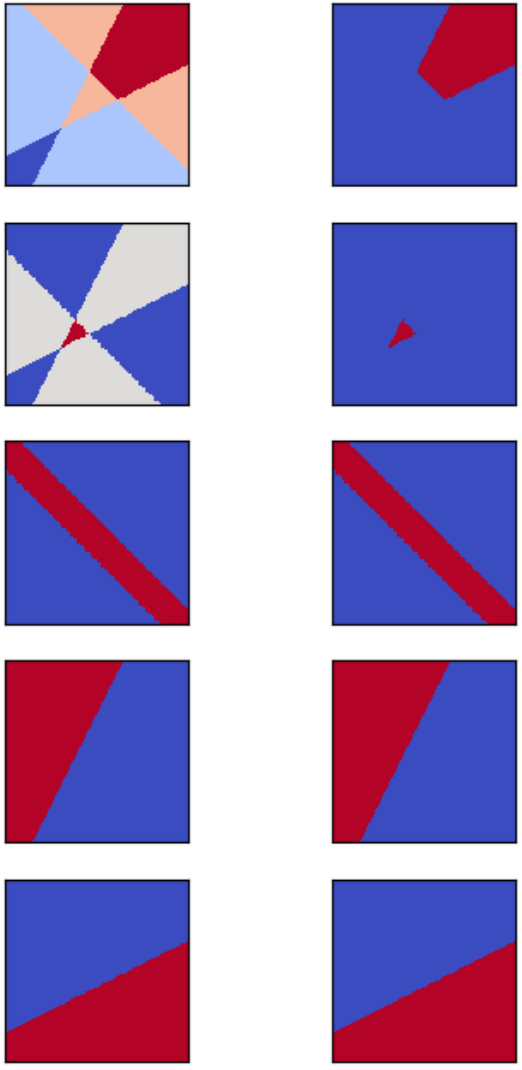
- $1 - N_{14} = 1$;
- $N_{13} = 1$;
- $N_{12} = 1$,

то в ней и только в ней $1 - N_{14} + N_{13} + N_{12} = 3$.

Тогда можно сконструировать её нейрон второго слоя, например, так:
$1 - N_{14} + N_{13} + N_{12} > 2.5$ (2.5, а не 3, чтобы не требовать идеальной точности от входных нейронов)
$N_{12} + N_{13} - N_{14} - 1.5 > 0$
$N_{21} = \mathbb{S}(N_{12} + N_{13} - N_{14} - 1.5)$

Аналогично получим другие нейроны второго слоя:
$N_{22} = \mathbb{S}(-N_{11} + N_{12} - N_{14} - 0.5)$
$N_{23} = \mathbb{S}(N_{11} - N_{13} - 0.5)$
$N_{24} = \mathbb{S}(N_{14} - 0.5)$
$N_{25} = \mathbb{S}(-N_{12} + 0.5)$

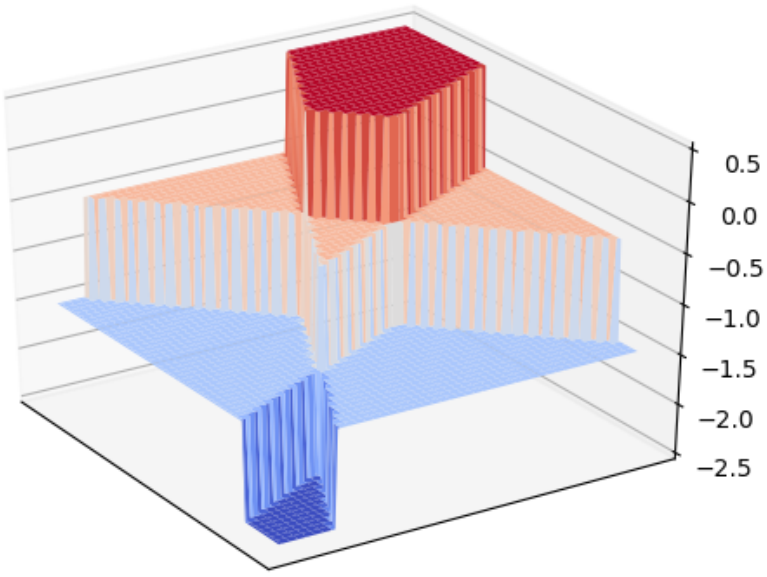
Выходы нейронов второго слоя

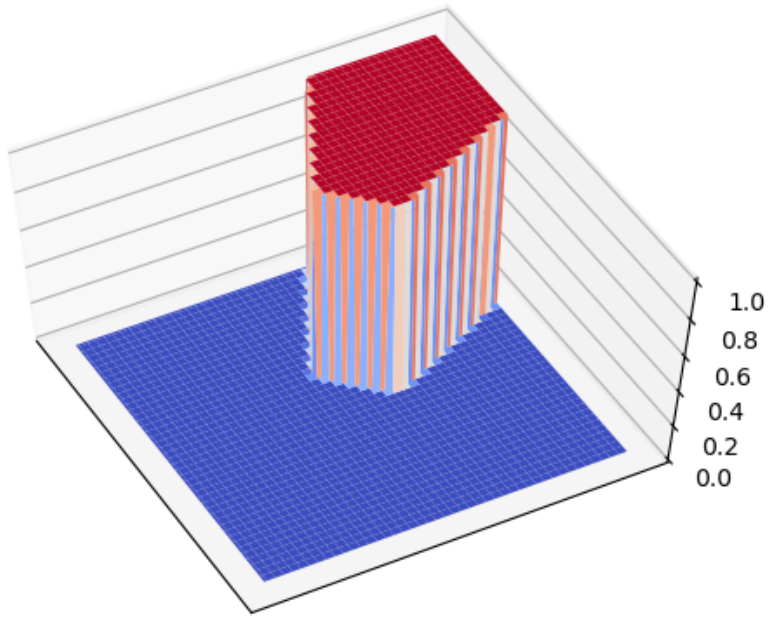


Графики построены для диапазонов x и y $[-3, 10]$.
Слева показаны взвешенные суммы каждого нейрона второго слоя до нелинейного преобразования ($\sum w_i x_i$), справа — нелинейные преобразования от них, т.е. выходы нейронов $\mathbb{S}(\sum w_i x_i)$.

Выходы нейронов второго слоя: детали

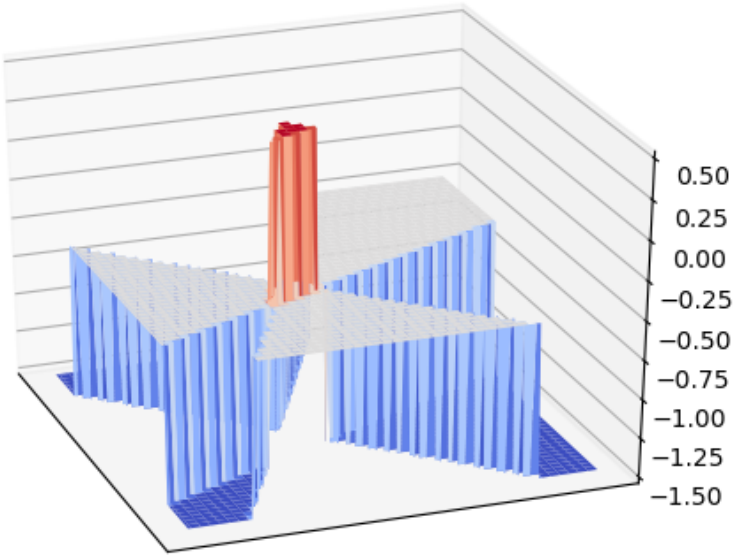
Выход первого нейрона до и после нелинейного преобразования:

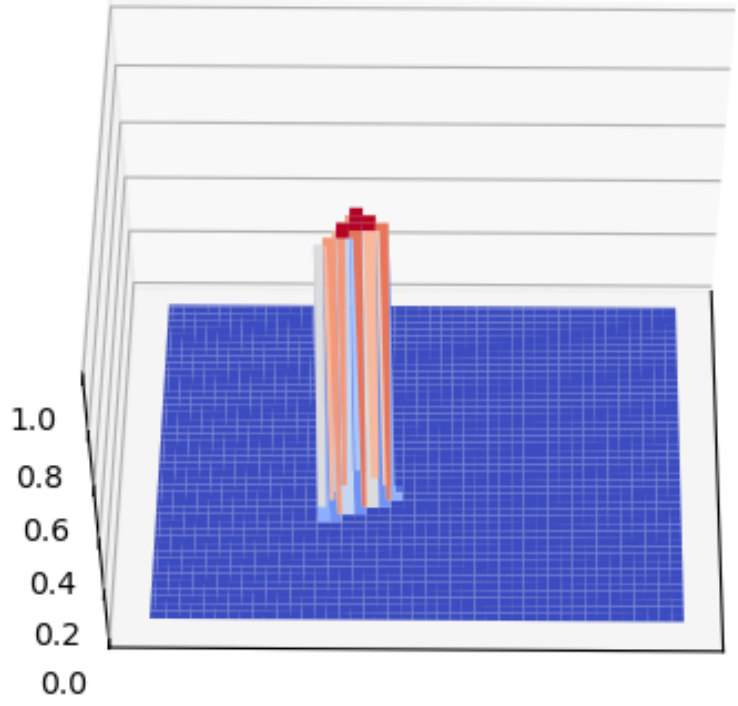




Выходы нейронов второго слоя: детали (2)

Выход второго нейрона до и после нелинейного преобразования:





Пример: третий слой

Третий (выходной) слой содержит комбинации выпуклых фигур, распознанных вторым слоем. Таких комбинаций две — распознаваемая фигура и остальные части плоскости.

Для распознаваемой фигуры надо, чтобы N_{21} **или** N_{22} были активны, а остальные — неактивны, для оставшейся части — наоборот.

Вопрос: почему на втором слое должны активироваться все входы, а на третьем — хотя бы один?

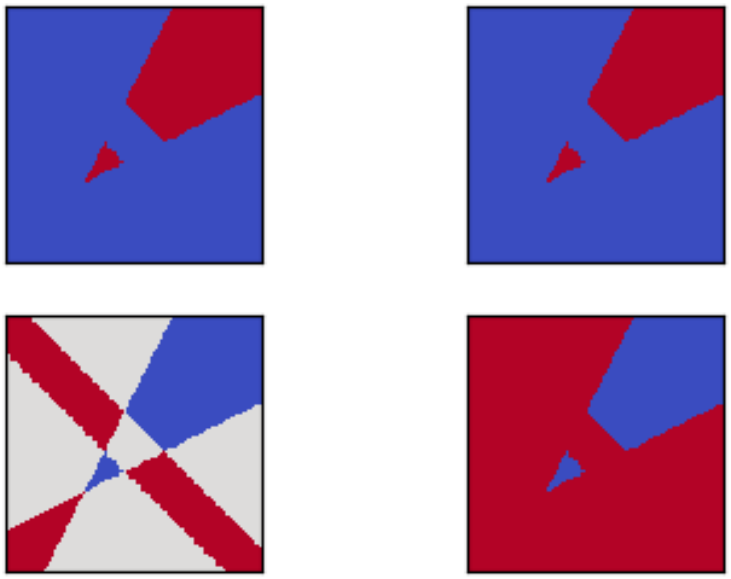
Пример: третий слой

Третий (выходной) слой содержит комбинации выпуклых фигур, распознанных вторым слоем. Таких комбинаций две — распознаваемая фигура и остальные части плоскости.

Для распознаваемой фигуры надо, чтобы N_{21} **или** N_{22} были активны, а остальные — неактивны, для оставшейся части — наоборот.
Вопрос: почему на втором слое должны активироваться все входы, а на третьем — хотя бы один?
Ответ: потому что на втором слое выполняется пересечение областей, а на третьем — объединение.

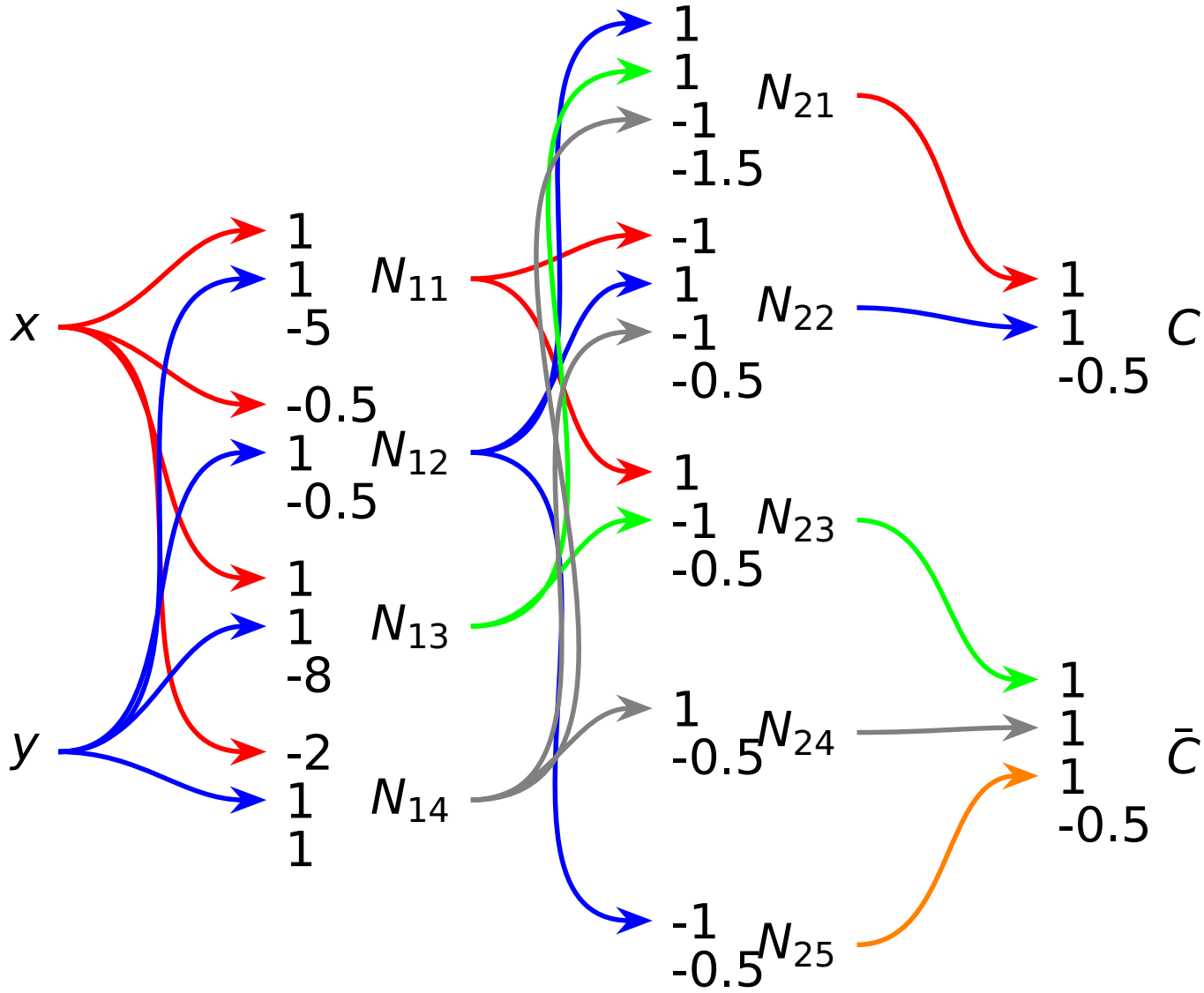
В распознаваемой области надо, чтобы хотя бы один из N_{21} и N_{22} был больше 0.5, т.е. чтобы $N_{21} + N_{22} > 0.5$, т.е. $N_{21} + N_{22} - 0.5 > 0$
Аналогично вне распознаваемой области надо, чтобы $N_{23} + N_{24} + N_{25} > 0.5$, т.е. $N_{23} + N_{24} + N_{25} - 0.5 > 0$. Добавляем два выходных нейрона по этим формулам.

Выходы сети



Графики построены для диапазонов x и y $[-3, 10]$. Слева показаны взвешенные суммы каждого нейрона выходного слоя до нелинейного преобразования ($\sum w_i x_i$), справа — нелинейные преобразования от них, т.е. выходы нейронов $\mathbb{S}(\sum w_i x_i)$.

Пример: структура итоговой сети



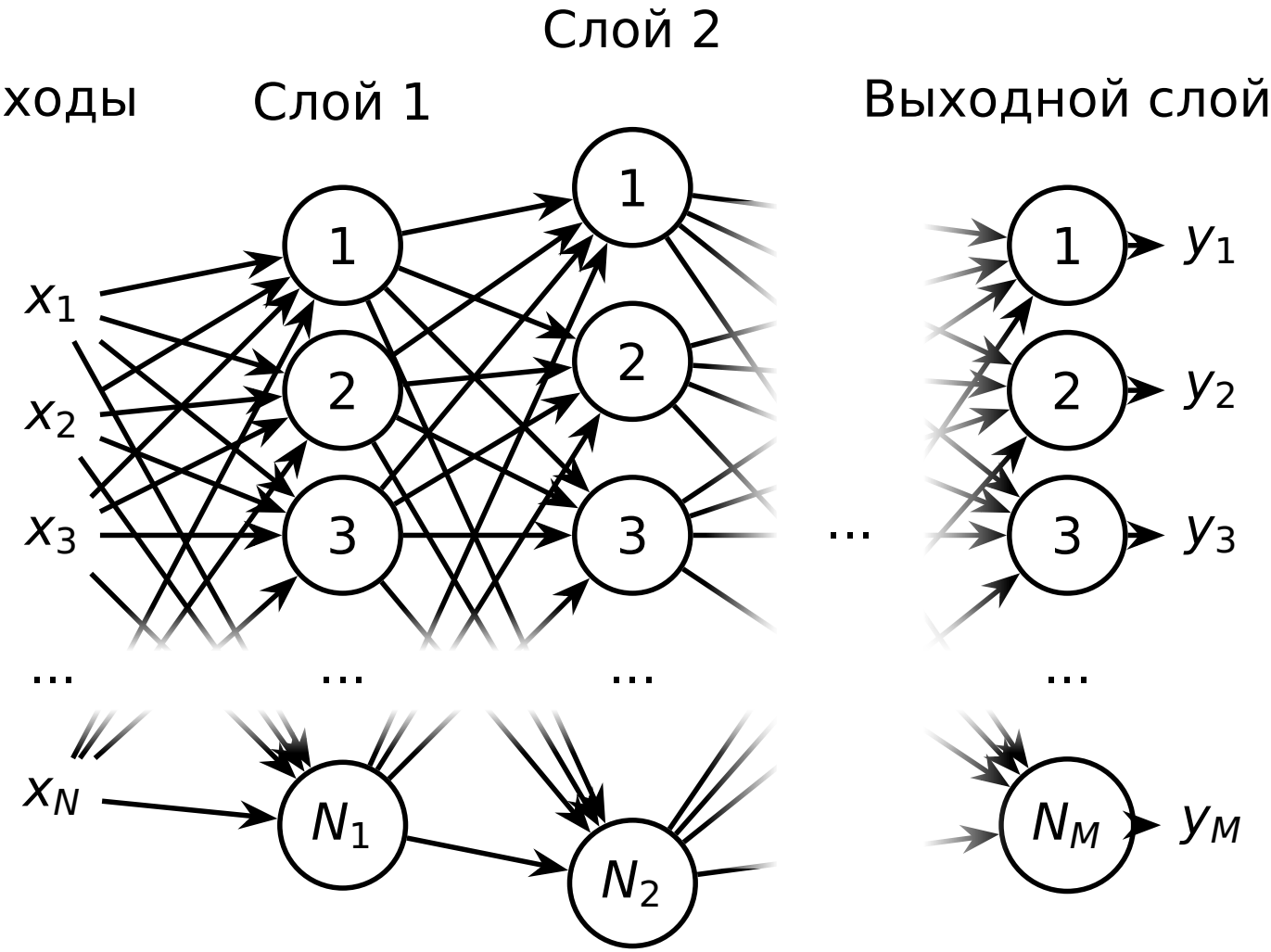
Универсальный аппроксиматор

Реально возможность реализации такой структуры зависит от формы функций активации, начального состояния сети и хода обучения, т.к. обучение — стохастический процесс. На практике часто требуется больше нейронов.

Многослойный персептрон (multi-layer perceptron, MLP) — комбинация нескольких последовательных слоёв без обратных связей. Это универсальный аппроксиматор, т.к. он способен при достаточно большом (но конечном) числе нейронов и слоёв приблизить любую непрерывную функцию.

Проблема: предсказать требуемое число нейронов и слоёв в общем случае невозможно; приходится либо пробовать разные варианты и выбирать наилучшие, либо руководствоваться накопленным опытом в данной предметной области.

Структура многослойного персептрона



Структура многослойного персептрона(2)

Обычно «входным слоем» считают сами переменные, хранящие выборку; все слои, кроме последнего — **скрытые**.

Обычно у всех нейронов одного слоя одна функция активации, хотя это не обязательно. У разных слоёв часто бывают разные функции активации. А вот синаптические коэффициенты w у каждого нейрона уникальны; в их подборе обычно заключается обучение многослойного персептрона.

Работа многослойного персептрона

1. на первый слой подаются признаки обрабатываемого прецедента $X = x_i, i = 1..N$;
2. выходы нейронов каждого следующего слоя рассчитываются по приведённому выше выражению для формального нейрона независимо друг от друга;
3. выходы последнего слоя интерпретируются в зависимости от смысла задачи.

Например, для классификации часто делают число нейронов в выходном слое равным числу классов; чем больше выход нейрона номер i, тем достовернее прецедент относится к соответствующему классу C_i.

Обучение многослойного персептрона

Кроме простейших случаев, единственный метод — градиентный спуск. Возникает проблема: как настраивать такое огромное количество синаптических коэффициентов?

Общий метод — **обратное распространение ошибок** (backpropagation). Идея: менять те коэффициенты, которые приводят к ошибкам конкретного нейрона. В отличие от нормального режима работы, при обучении вычисления идут от последнего слоя к первому.

Обучение одного слоя

Пусть персептрону предъявлен некий единственный прецедент, и результат — вектор выходов нейронов последнего слоя — отличается от ожидаемого (верного). Пусть на предпоследнем слое M нейронов, а на последнем — N нейронов вида

$$y_i = \varphi_i(\sum_{j=0}^{M} w_{ij}x_j), i = 1..N$$

то есть у каждого из них своя функция активации φ_i и свои синаптические коэффициенты $w_{ij}, j = 1..M$ для обработки выхода предпоследнего слоя $X = \{1, x_1, x_2, ..., x_M\}$.

Хотелось бы минимизировать сумму квадратов ошибок:

$$L = \sum_{i=1}^{N}(y_i - a_i)^2,$$

где a_i — верное значение выхода нейрона i.

Обучение одного слоя: градиентный спуск

Чтобы совершить спуск, нужно посчитать частные производные суммы квадратов ошибок по синаптическим коэффициентам w_{ij}:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial w_{ij}} = 2(y_i - a_i)\frac{\partial y_i}{\partial w_{ij}},$$

причём $(y_i - a_i) = \varepsilon_i$ — ошибка нейрона i.

$$\frac{\partial y_i}{\partial w_{ij}} = \varphi_i'(\sum_{j=0}^{M} w_{ij}x_j)x_j$$

Иначе говоря, **величина градиента по синаптическому коэффициенту** w_{ij} **пропорциональна** $\varepsilon_i\varphi_i'x_j$

Выводы по градиентному спуску

Что значит «градиент пропорционален $\varepsilon_i \varphi_i' x_j$»?

- Чем больше ошибка выходного нейрона, тем больше градиент;
- чем круче функция активации выходного нейрона, тем больше градиент;
- чем больше пришедшее на вход значение, тем больше градиент.

Тогда для коррекции ошибки можно поменять коэффициент w_{ij} на величину:

$$\delta w_{ij} = -\alpha \varepsilon_i \varphi_i' x_j,$$

где α — константа (**скорость обучения**).

Другие слои

Как рассчитать ошибку других слоёв, ведь нам неизвестны их требуемые значения?

Самый распространённый метод: раз $\frac{\partial L}{\partial y_i}$ — ошибка на последнем слое, назовём ошибкой на скрытом слое $\frac{\partial L}{\partial x_j}$:

$$\frac{\partial L}{\partial x_j} = 2\sum_{i=1}^{N}(y_i - a_i)\frac{\partial y_i}{\partial x_j} = 2\sum_{i=1}^{N}\varepsilon_i\varphi_i' w_{ij}$$

После этого можно применить ту же логику к предпоследнему слою и т.п. до первого.

Свойства обратного распространения ошибок

Широко применяется на практике благодаря универсальности и, как правило, хорошим результатам.

Недостатки — те же, как у всех методов градиентного спуска: итеративность, локальные минимумы, медленная сходимость, общая непредсказуемость поведения и т.п.

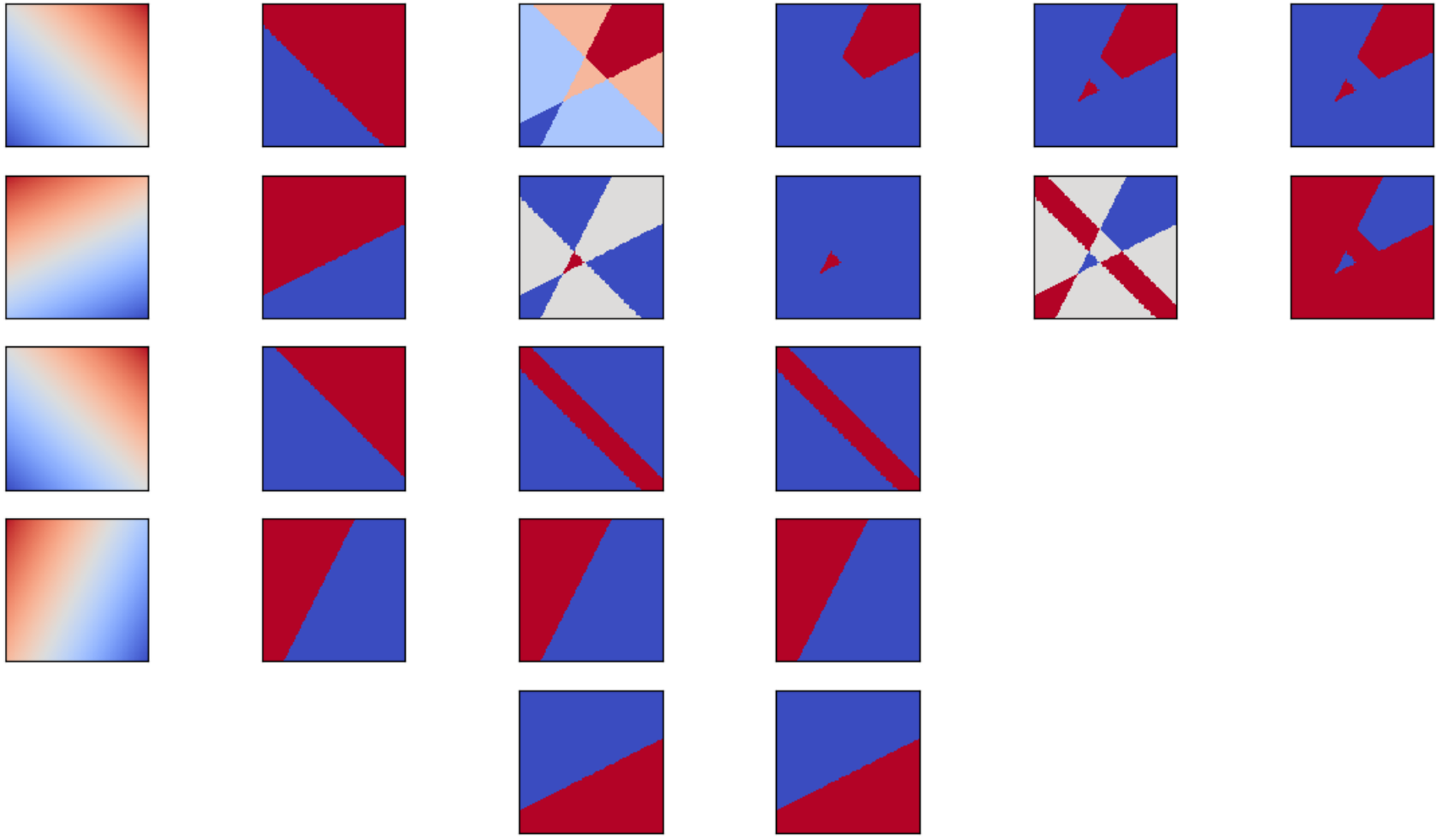
Для улучшения свойств часто применяют метод **имитации отжига**: скорость обучения сначала делают высокой, а потом снижают, чтобы избавиться от застреваний в локальном минимуме в самом начале и получить более предсказуемое поведение в конце: $\alpha_n = \alpha_0/n$, где n — номер итерации.

Проблема: число нейронов плохо предсказывается

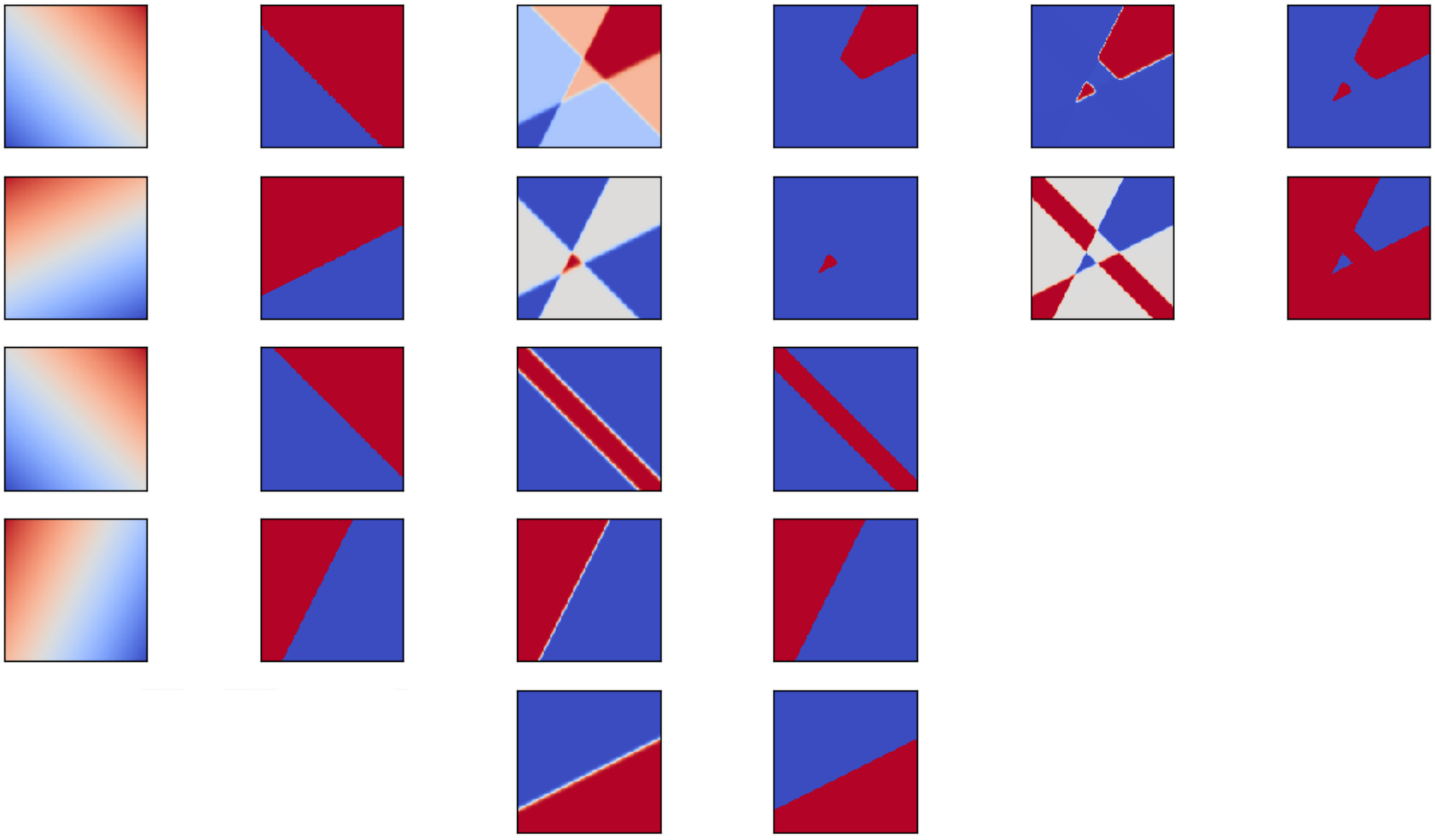
Ранее мы смогли точно сказать число нейронов, нужное для точной классификации. В реальности так точно оценить его обычно не получается: для работы обратного распространения ошибок нелинейная функция должна быть достаточно гладкой, что затрудняет чёткое отделение областей.

Посмотрим, что будет, если заменить в нашей сети ступень $\mathbb{S}$ (разрывную функцию) на логистическую.

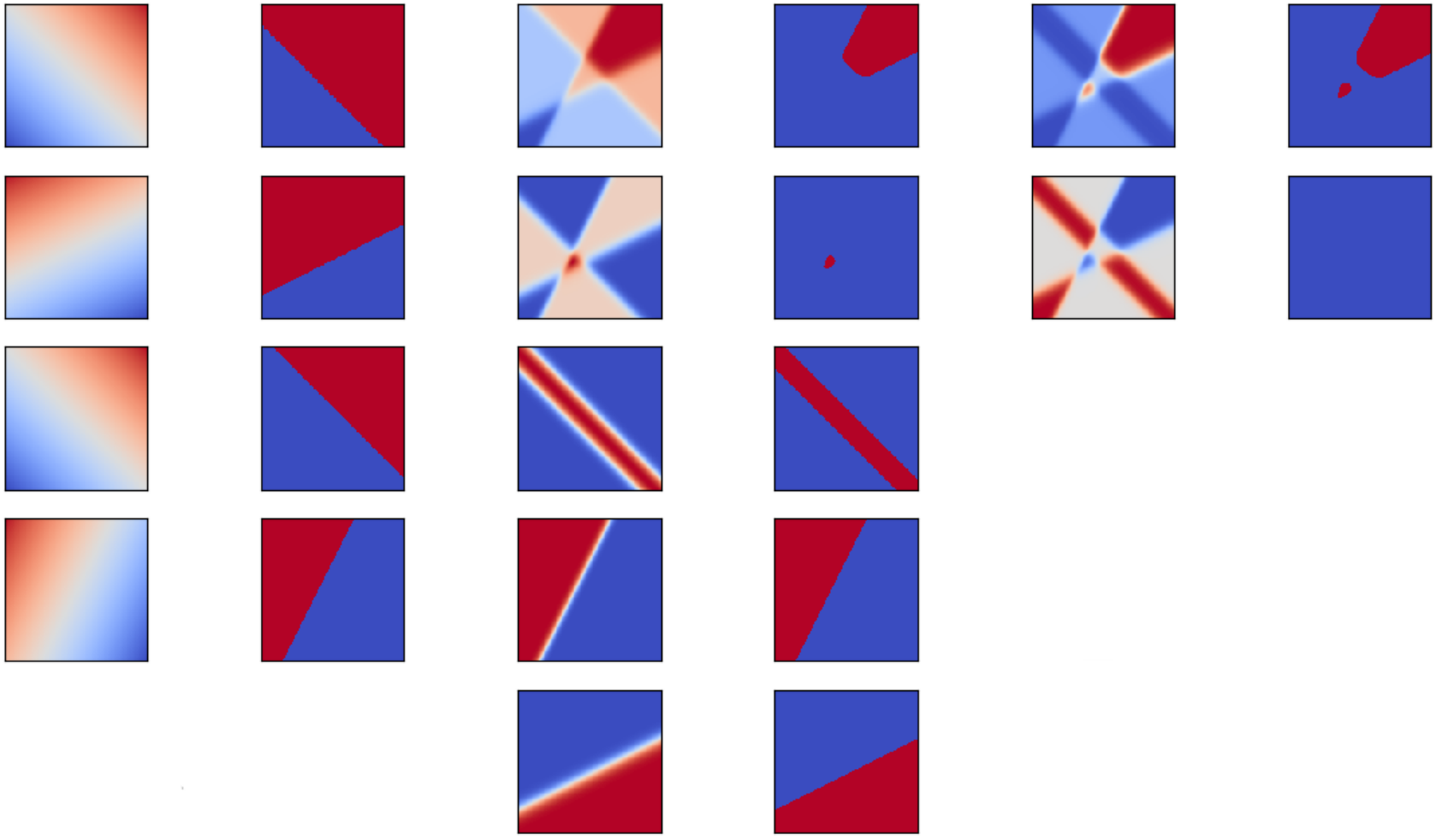
Выходы сети для ступенчатой функции активации



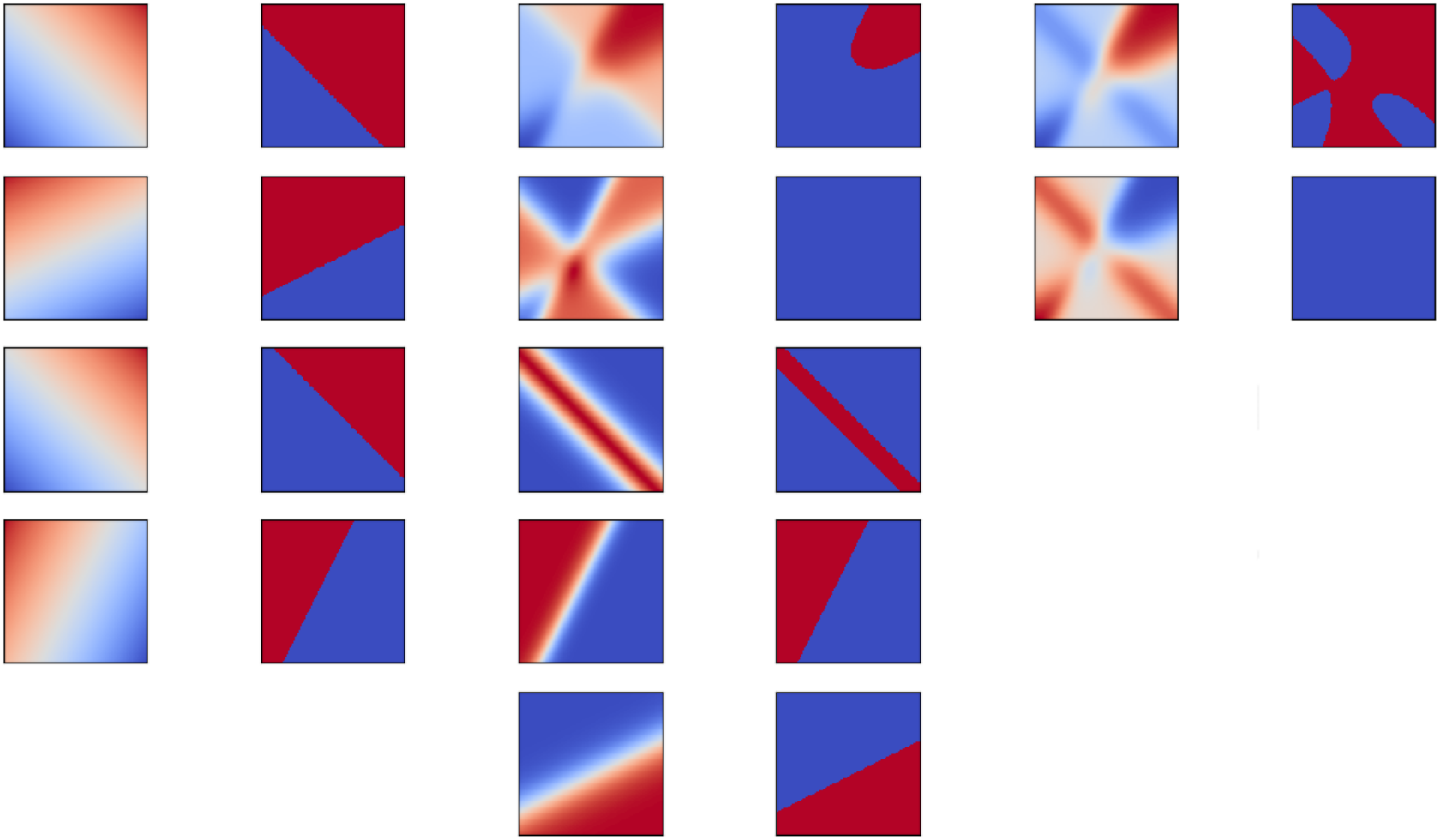
Выходы сети для логистической функции активации с s=10



Выходы сети для логистической функции активации с s=3

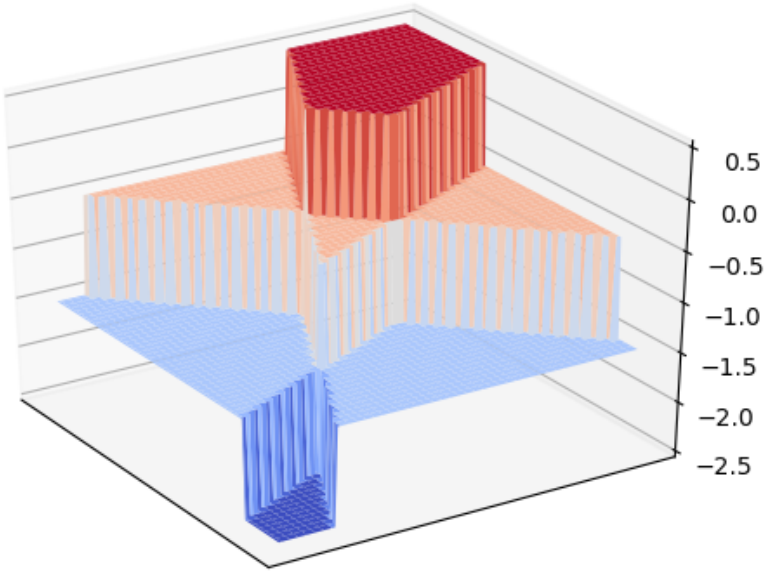


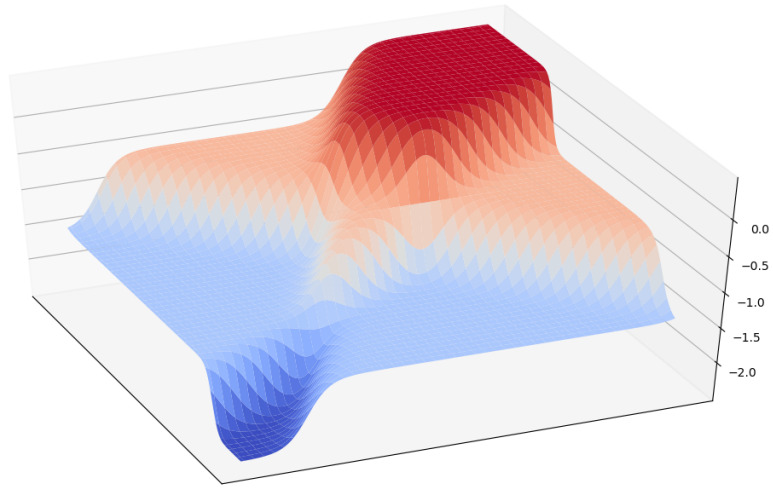
Выходы сети для логистической функции активации с s=1



Изменения в выходах нейронов

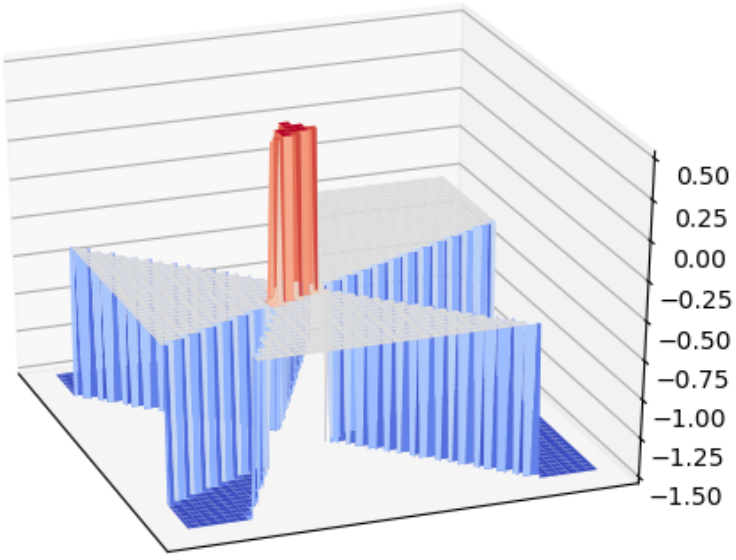
Изменения в первом нейроне второго слоя: слева — сумма при ступенчатой функции активации, справа — при логистической функции активации с s=3.

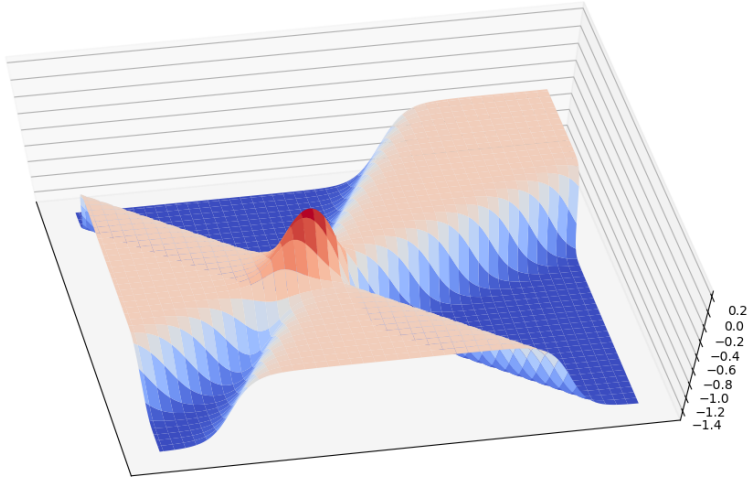




Изменения в выходах нейронов

Изменения во втором нейроне второго слоя: слева — сумма при ступенчатой функции активации, справа — при логистической функции активации с s=3.

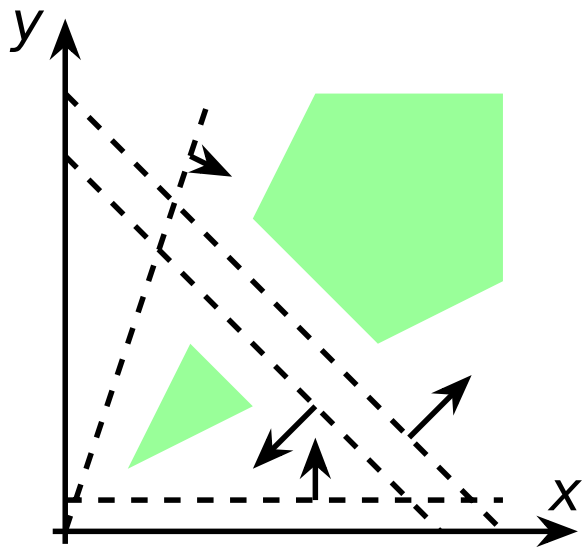




Вывод: функции активации влияют на обобщающую способность сети; предсказать степень этого влияния сложно, поэтому оценить нужное число нейронов и слоёв трудно.

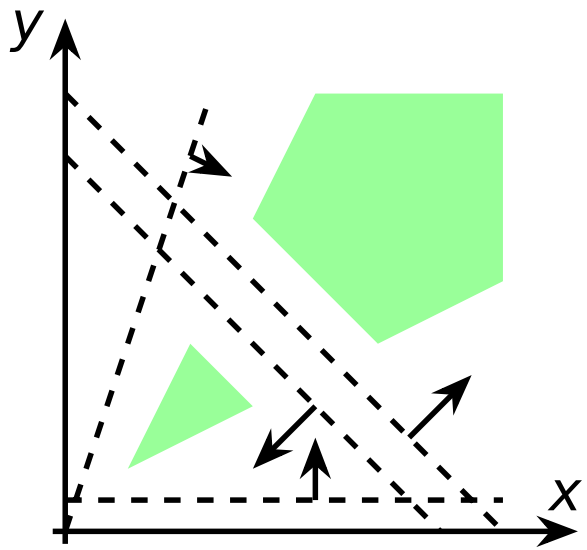
Влияние начальных условий

Вопрос: если пороги функций активации четырёх нейронов первого слоя случайно распределились так, как показано на рисунке, велик ли шанс, что классификация будет точной?



Влияние начальных условий

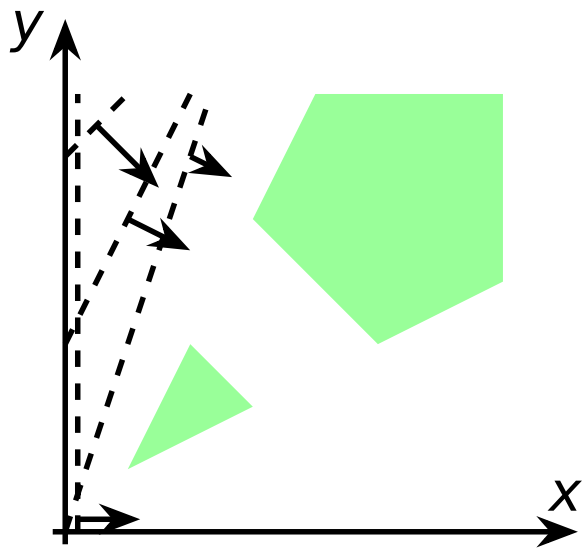
Вопрос: если пороги функций активации четырёх нейронов первого слоя случайно распределились так, как показано на рисунке, велик ли шанс, что классификация будет точной?



Ответ: да, потому что они вполне могут спуститься по градиенту к простейшему правильному решению.

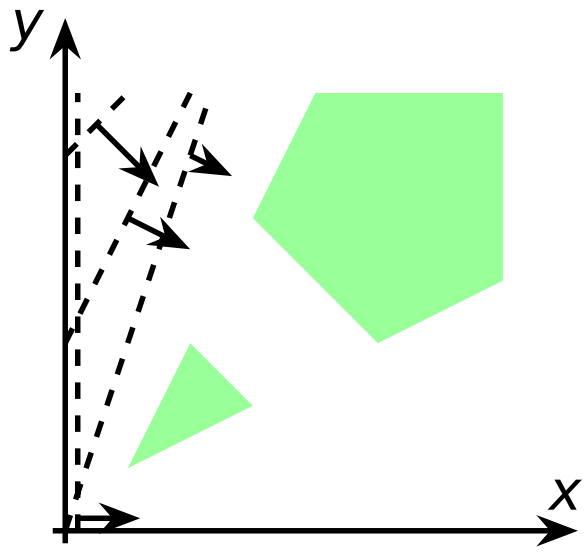
Влияние начальных условий(2)

Вопрос: а при таком расположении начальных порогов?



Влияние начальных условий(2)

Вопрос: а при таком расположении начальных порогов?



Ответ: шансов меньше, потому что нейроны будут спускаться по слишком близким траекториям и скорее всего попадут в локальный минимум.

Гарантий нет ни в одном случае, потому что неизвестны параметры нейронов второго и третьего слоёв, а также скорость обучения и распределение примеров выборки по пространству.

Другие сети

Позже отдельно поговорим о глубоких сетях (потому что их удобнее рассматривать в конкретных применениях) и сетях Кохонена (потому что ими обычно не решают задачи классификации).

Кроме сетей с прямым распространением, есть много видов реккурентных сетей, в которых нейроны соединены циклически. Рассмотрим пока только сеть Хопфилда.

Задача запоминания ассоциаций

Идея: заставить сеть запомнить несколько образов (сочетаний выходов) таким образом, что при предъявлении какого-либо входа сеть меняет состояние на один из этих сохранённых образов.

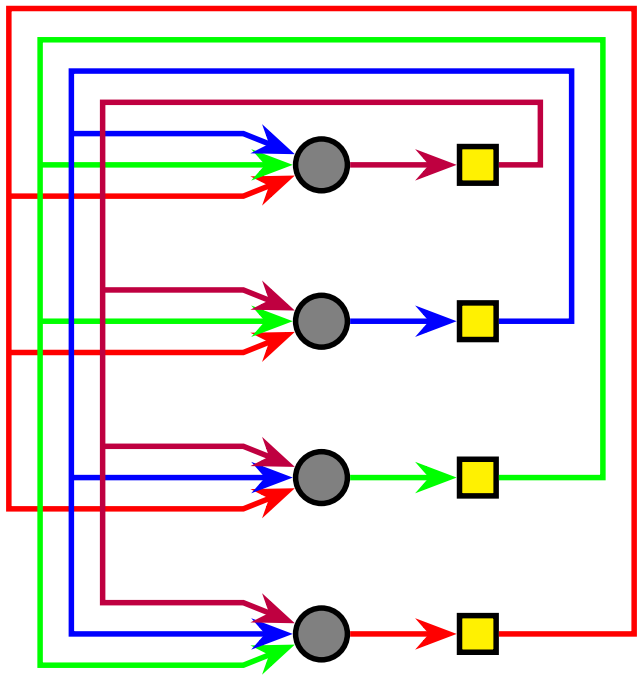
Например, пусть сеть из 4 нейронов благодаря некоторому сочетанию синаптических коэффициентов и обратных связей между нейронами способна устойчиво находиться только в следующих состояниях: 0101, 1111, 1010. Это значит, что при попытке ввести нейроны в другое сочетание (например, 1100) сеть будет переключаться до тех пор, пока не перейдёт в одно из трёх устойчивых состояний.

Если обеспечить, чтобы сеть переключалась в состояние, наиболее похожее на предъявленное, она сумеет «вспомнить» образ, если предъявить его в несколько искажённом виде.

Сеть Хопфилда

Одна из предложенных физиологически обоснованных моделей ассоциативной сети — сеть Хопфилда:

- серым обозначены нейроны; жёлтым — элементы, запоминающие текущие выходы нейронов для подачи их на другие нейроны (т.е. элементы задержки).
- значение нейрона не подаётся само на себя.
- все входы и выходы имеют устойчивые значения только 1 или -1.



Коэффициенты

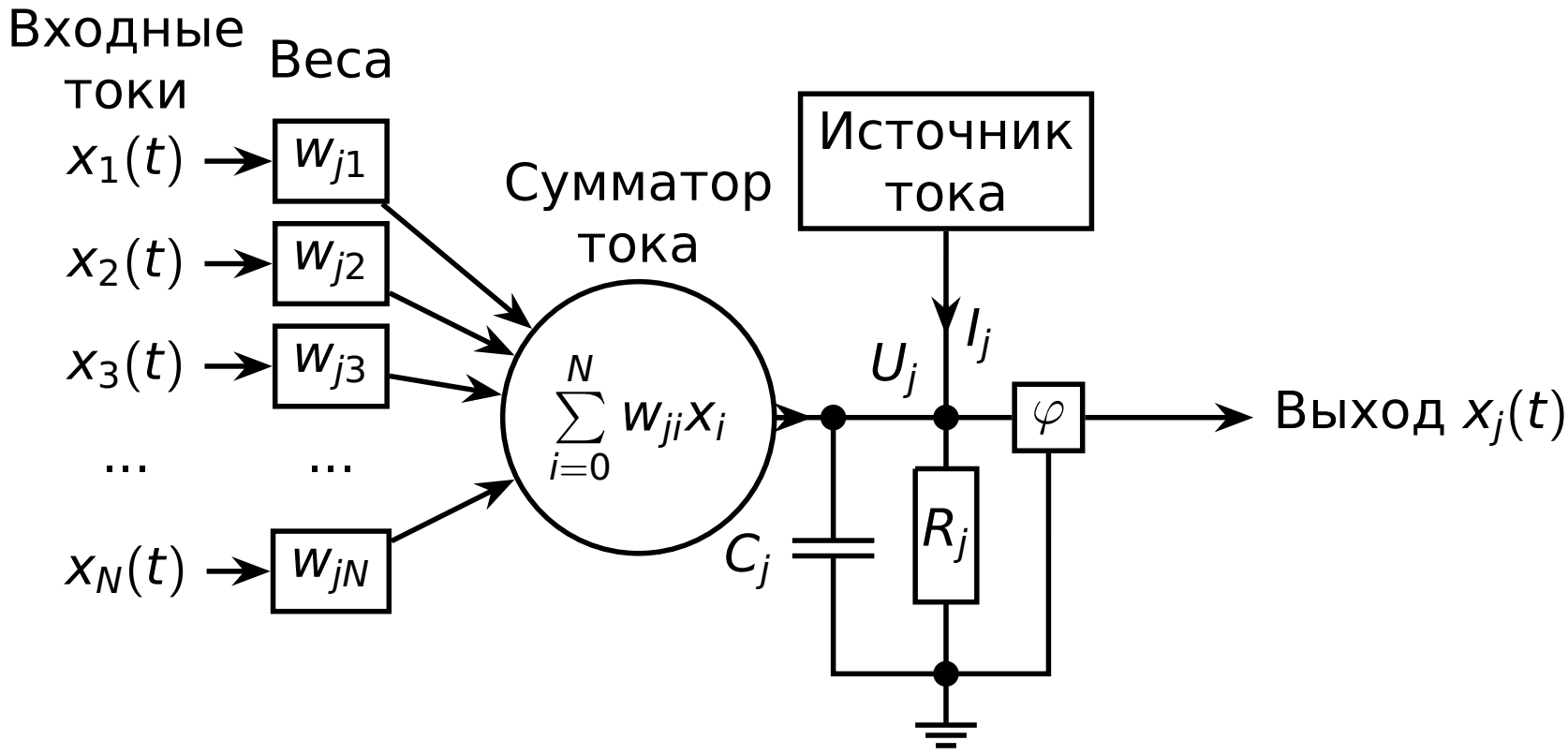
Матрица коэффициентов $\mathbb{W} = \{w_{ij}\}$ — квадратная, имеет нулевую диагональ (так как нейроны не влияют на себя).

При подаче на входы некоторого значения сеть начинает переключаться (менять выходы при необходимости) и делает это, пока не достигнет устойчивого состояния. Это значит, что для запоминаемых образов X верно равенство $X = \varphi(\mathbb{W}X)$

Получить коэффициенты для сохранения векторов $p_i, i = 1..K$ можно по выражению $\frac{1}{N}\sum_{i=1}^{K} pp^T$, где N — число нейронов.

Аддитивная модель нейрона

Простейшая модель нейрона с памятью для формального обоснования работы сети:



По законам Кирхгофа:

$$C_j \frac{dU_j(t)}{dt} + \frac{U_j(t)}{R_j} = \sum_{i=1}^{N} w_{ji}x_i(t) + I_j$$

Формальное обоснование: уравнение динамики

Тогда дифференциальное уравнение системы имеет вид:

$$C_j \frac{dU_j(t)}{dt} = -\frac{U_j(t)}{R_j} + \sum_{i=1}^{N} w_{ji} x_i(t) + I_j$$

Так как $x_i(t) = \varphi_i(U_i(t))$, то

$$C_j \frac{dU_j(t)}{dt} = -\frac{U_j(t)}{R_j} + \sum_{i=1}^{N} w_{ji} \varphi_i(U_i(t)) + I_j$$

Если матрица $\mathbb{W}$ симметрична, функции активации нейронов известны и обратимы, то можно доказать, что такая система найдёт минимум энергии (функции Ляпунова), т.е. сеть Хопфилда ассимптотически устойчива.

Формальное обоснование: функция Ляпунова

Хопфилд попытался использовать как функцию Ляпунова данной системы следующую функцию:

$$E = -\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} w_{ji}x_i x_j + \sum_{j=1}^{N}\frac{1}{R_j}\int_0^{x_j}\varphi_j^{-1}(x)dx - \sum_{j=1}^{N} I_j x_j$$

Её производная по времени равна

$$\frac{dE}{dt} = -\sum_{j=1}^{N}(\sum_{i=1}^{N} w_{ji}x_i - \frac{U_j}{R_j} + I_j)\frac{dx_j}{dt} = -\sum_{j=1}^{N} C_j \frac{dU_j}{dt}\frac{dx_j}{dt},$$

так как $x_j = \varphi(U_j)$

Формальное обоснование: функция Ляпунова (2)

$$\frac{dE}{dt} = -\sum_{j=1}^{N} C_j \frac{dU_j}{dt} \frac{dx_j}{dt},$$

Так как функции активации нейронов обратимы, $U_j = \varphi^{-1}(x_j)$ и

$$\frac{dE}{dt} = -\sum_{j=1}^{N} C_j \frac{d\varphi^{-1}(x_j)}{dt} \frac{dx_j}{dt} = -\sum_{j=1}^{N} C_j \frac{d\varphi^{-1}(x_j)}{dx_j} (\frac{dx_j}{dt})^2$$

Обратная функция от функции активации неубывающая, поэтому $\frac{d\varphi^{-1}(x_j)}{dx_j} \geq 0$. Также $(\frac{dx_j}{dt})^2 \geq 0$. Это значит, что

$$\frac{dE}{dt} \leq 0,$$

т.е. энергия монотонно невозрастает. Так как она ограничена, то 1) E — функция Ляпунова и 2) модель устойчива.